

Atividade de Fixação POO, Try, Classes, Classes Abstratas e Atributos Privados

Questão 1: Criando uma classe com atributos privados

Crie uma classe chamada `ContaBancaria` com os seguintes atributos privados: `saldo` e `titular`.

Adicione métodos `get_saldo`, `set_saldo`, `get_titular` e `set_titular`.

O método `set_saldo` deve permitir adicionar saldo apenas se o valor for positivo. Caso contrário, levante uma exceção `ValueError` e trate-a com `try-except`.

Questão 2: Classe abstrata de veículo

Crie uma classe abstrata chamada `Veiculo` que contém o método abstrato `mover()`.

Implemente duas classes concretas, `Carro` e `Bicicleta`, que herdam de `Veiculo` e implementam o método `mover()`.

Use `try-except` para tratar erros ao tentar criar uma instância de `Veiculo`.

Questão 3: Gerenciando usuários com atributos privados

Implemente uma classe chamada `Usuario` com atributos privados `nome` e `email`.

Crie métodos `get` e `set` para ambos.

Use `try-except` para evitar que um email inválido (sem "@" ou ".com") seja atribuído ao atributo `email`.

Questão 4: Cadastro de produtos com preço válido

Crie uma classe chamada `Produto` que possui os atributos privados `nome` e `preco`.

Implemente os métodos `get` e `set`. O preço deve ser maior que zero; caso contrário, levante um erro tratado com `try-except`.

Questão 5: Sistema de login

Implemente uma classe `Login` com atributos privados `usuario` e `senha`.

Use métodos `get` e `set` e adicione validação na senha (deve ter pelo menos 6 caracteres). Use `try-except` para tratar erros de senha inválida.

Questão 6: Classe abstrata para contas bancárias

Crie uma classe abstrata `Conta` com métodos abstratos `depositar` e `sacar`.

Implemente as classes concretas `ContaCorrente` e `ContaPoupanca` que herdam de `Conta` e implementam os métodos.

Detalhes

- A classe `ContaCorrente` deve permitir saques ilimitados, mas cobrar uma taxa de R\$ 2.50 por saque.
- A classe `ContaPoupanca` deve permitir apenas 3 saques gratuitos por mês; após isso, cada saque terá uma taxa de R\$ 5.00.

Questão 7: Cadastro de empregados

Crie uma classe `Empregado` com atributos privados `nome` e `salario`.

Adicione validação para que o salário seja maior que zero. Caso contrário, levante uma exceção e trate com `try-except`.

Questão 8: Gerenciamento de estoque

Implemente uma classe `Estoque` para gerenciar produtos, incluindo métodos para adicionar e remover produtos.

Use validação para impedir remoção se o estoque estiver vazio e trate o erro com `try-except`.

Questão 9: Sistema de notas

Crie uma classe `Aluno` com atributos `nome` e `nota`.

Valide para que a nota seja entre 0 e 10. Caso contrário, levante uma exceção e trate com `try-except`.

Questão 10: Herança com validação

Crie uma classe `Funcionario` que herda de uma classe base `Pessoa`.

Adicione validação no atributo `salario` (deve ser positivo) e use `try-except` para lidar com erros durante a atribuição.